Simulación de Ecosistema Predador-Presa

Reporte Técnico de Arquitectura, Implementación Secuencial y Paralelización con POSIX Threads (Pthreads)

Proyecto: Asignación de Programación Paralela (MP3)
Tema: Autómata Celular y Modelo Lotka-Volterra Concurrente
Modelo de Hilos: POSIX Threads (Pthreads)
Configuración: 4 Hilos Fijos (Thread Pool Persistente)

1. Introducción y Descripción del Problema

El presente proyecto comprende el modelado, diseño e implementación de un simulador estocástico y determinista de un ecosistema virtual bidimensional basado en interacción de especies (*predador-presa*). El espacio de simulación consiste en una grilla de R filas por C columnas donde cada celda puede estar vacía o albergar uno de los siguientes tres elementos:

- `ROCK` **Rocas:** Elementos ambientales fijos e inmutables que no se desplazan ni pueden ser ocupados por ninguna criatura. Proporcionan estructura y refugio en el ecosistema.
- `RABBIT` **Conejos:** Especie herbívora de reproducción rápida. Se desplazan hacia celdas vacías adyacentes y procrean cada `GEN_PROC_RABBITS` generaciones al desplazarse. Sirven de presa para los zorros.
- `FOX` **Zorros:** Depredadores carnívoros. Intentan alimentarse de conejos adyacentes; si no encuentran alimento y superan `GEN_FOOD_FOXES` generaciones sin comer, mueren por inanición. Si sobreviven, buscan celdas vacías y procrean tras `GEN_PROC_FOXES` generaciones.

1.1. Reglas de Desplazamiento y Criterio de Selección

Los animales se mueven en 4 direcciones ortogonales (no diagonales) evaluadas estrictamente en **orden horario**: **Norte (0)**, **Este (1)**, **Sur (2)** y **Oeste (3)**. Si para una entidad en la posición (X, Y) durante la generación G existen P celdas válidas a las que desplazarse, la celda destino se escoge unívocamente mediante la fórmula:

$$\text{Índice Elegido} = (G + X + Y) \pmod P$$

Si no existen celdas admisibles ($P = 0$), el animal permanece en su ubicación actual.

1.2. Resolución de Conflictos y Simultaneidad

Todos los animales de una especie actúan de manera simultánea en su fase correspondiente:

- **Conflicto Conejo - Conejo:** Si dos o más conejos convergen en la misma celda destino, sobrevive únicamente aquel con la **mayor edad de procreación** (`proc_age`). Todos los demás conejos mueren.
- **Conflicto Zorro - Conejo:** El zorro devora al conejo presente en la celda destino. El conejo perece inmediatamente y el contador de hambre del zorro se restablece (`starve_age = 0`).

- **Conflicto Zorro - Zorro:** Si dos o más zorros compiten por la misma celda, sobrevive aquel con **mayor edad de procreación**; en caso de empate, sobrevive el **menos hambriento** (menor `starve_age`). Si el zorro derrotado había devorado un conejo en ese turno, transfiere el crédito alimenticio al zorro superviviente.

2. Arquitectura de Software y Diseño Modular

El sistema fue concebido bajo el principio de alta cohesión y bajo acoplamiento. Se estableció como premisa fundamental que **únicamente el método `Simulator::update()` fuera modificado para la versión paralela**, manteniendo intacta la interfaz pública y el comportamiento de los demás componentes.

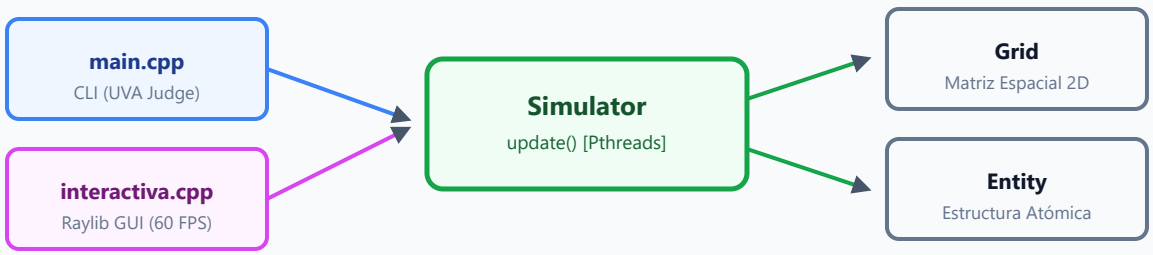


Figura 1. Arquitectura modular desacoplada del sistema de simulación.

- **`Entity` (`entity.h`):** Estructura plana de datos que modela el tipo de objeto, identificador único, edades (reproducción y hambre), estado de vida y coordenadas.
- **`Grid` (`grid.h` , `grid.cpp`):** Matriz bidimensional dinámica (`std::vector<std::vector<Entity*>>`) con indexación directa mediante `operator[]` . Las celdas libres se representan de forma eficiente mediante punteros nulos (`nullptr`).
- **`Simulator` (`simulator.h` , `simulator.cpp`):** Gestor del estado global, estadísticas acumuladas, doble búfer de memoria (`grid` actual y `next_grid` de destino) y motor de actualización temporal.
- **Consumidores:** `main.cpp` procesa entradas y salidas de texto estándar estilo juez UVA, mientras que `interactiva.cpp` ofrece visualización fluida con gráficos en tiempo real mediante la librería Raylib.

3. Implementación Secuencial

En la versión secuencial, cada generación evoluciona de manera determinista ejecutando un pipeline de 8 etapas lineales sobre un esquema de **Double Buffering**:

1. **Copia de Rocas:** Se replican las rocas en `next_grid` en sus coordenadas inmutables (X, Y).
2. **Planificación de Conejos:** Cada conejo vivo consulta `grid` para identificar celdas vacías adyacentes y calcula su celda destino mediante $(G + X + Y) \pmod P$.
3. **Arbitraje de Conejos:** Se ubican los conejos en `next_grid`. Si dos conejos coinciden en una celda, sobrevive aquel con mayor `proc_age` y el perdedor se marca con `is_alive = false`.
4. **Reproducción y Envejecimiento de Conejos:** Los conejos que cambiaron de posición y alcanzaron `GEN_PROC_RABBITS` reinician su edad a 0 e insertan una nueva cría en su posición previa $(prev_x, prev_y)$. Los supervivientes que no procrearon incrementan su `proc_age`.
5. **Detección e Inanición de Zorros:** Cada zorro vivo busca conejos en `next_grid`. Si no encuentra conejos, comprueba si su inanición alcanza `GEN_FOOD_FOXES`: de ser así, **el zorro muere de hambre antes de poder moverse**. Si no muere, busca celdas libres y calcula su destino.
6. **Depredación y Arbitraje de Zorros:** Si la celda destino contiene un conejo en `next_grid`, el conejo muere devorado y el zorro marca `just_ate = true`. Si dos zorros convergen, sobrevive el de mayor `proc_age` (o menor `starve_age` ante empate).
7. **Reproducción, Hambre y Envejecimiento de Zorros:** Los zorros que se movieron y cumplen `GEN_PROC_FOXES` dejan una cría en su posición previa. Si comieron, reinician su hambre a 0; caso contrario, incrementan `starve_age++`.
8. **Compromiso y Swap:** Se destruye el antiguo `grid`, se promueve `next_grid` como la grilla activa, se instancia un nuevo `next_grid` y se incrementa el contador de generación `step++`.

4. Implementación Paralela con POSIX Threads (Pthreads)

4.1. Fundamentación y Estrategia de Paralelización

Para cumplir con el mandato de paralelizar exclusivamente `Simulator::update()` con máxima eficiencia, se adoptó un esquema híbrido de **Descomposición de Entidades en Fases Sincronizadas** apoyado en un **Thread Pool persistente de 4 hilos fijos**:

- **Eliminación de Sobrecarga de Creación:** Crear y destruir hilos con `pthread_create` / `pthread_join` en cada generación genera miles de llamadas al sistema y cambios de contexto innecesarios. Al instanciar los 4 hilos una sola vez en el constructor de `Simulator`, los hilos se mantienen latentes esperando señales de sincronización.
- **Fase de Lectura Libre de Locks:** El cálculo de desplazamientos de conejos lee únicamente `grid`, que es de solo lectura durante esa fase. Por ende, los 4 hilos procesan subrangos balanceados $[\frac{t \cdot N}{4}, \frac{(t+1) \cdot N}{4})$ con paralelismo masivo sin colisiones.

- **Barrera Inter-especie Obligatoria:** Los zorros deben cazar sobre conejos cuyo movimiento y arbitraje ya está 100% resuelto y plasmado en `next_grid`. Esto requiere sincronización estricta entre especies.

4.2. Primitivas de Sincronización Concurrente

A. Barrera Reusable con Variables de Condición (`PthreadBarrier`)

Para garantizar máxima portabilidad en cualquier entorno POSIX (evitando posibles carencias de `pthread_barrier_t` en plataformas específicas), se implementó una barrera basada en el algoritmo *Sense-Reversing Cycle* con `pthread_mutex_t` y `pthread_cond_t` :

```
void PthreadBarrier::wait() {
    pthread_mutex_lock(&this->mutex);
    std::size_t c = this->cycle;
    this->count++;
    if (this->count == this->total) {
        this->cycle++;
        this->count = 0;
        pthread_cond_broadcast(&this->cond);
    } else {
        while (c == this->cycle) {
            pthread_cond_wait(&this->cond, &this->mutex);
        }
    }
    pthread_mutex_unlock(&this->mutex);
}
```

Se estructuraron 3 barreras especializadas:

- `start_barrier` (capacidad 5: 4 workers + main): Despierta a los trabajadores al invocar `update()` .
- `step_barrier` (capacidad 4: solo workers): Coordina el avance unísono de los 4 hilos entre fases.
- `done_barrier` (capacidad 5: 4 workers + main): Notifica al hilo principal que el trabajo concurrente concluyó.

B. Exclusión Mutua Granular por Filas (`row_mutexes`)

Para escribir en `next_grid` sin carreras de datos y sin degradar el paralelismo con un cuello de botella global, se construyó un arreglo dinámico de R mutexes (uno por cada fila de la matriz). Al momento de arbitrar un destino (new_x, new_y), el hilo adquiere exclusivamente `row_mutexes[new_y]` :

> **Ventaja de Concurrencia:** Dos o más entidades que se desplazan a celdas situadas en filas distintas se procesan de forma **completamente paralela y simultánea**. Únicamente se produce contención cuando dos animales convergen en la misma fila.

C. Eliminación de Data Races y False Sharing (`WorkerContext`)

Modificar concurrentemente vectores dinámicos como `this->entities` o actualizar contadores globales provocaría corrupción de memoria o invalidación de líneas de caché (*False Sharing*). Cada hilo t cuenta con su propia instancia de `WorkerContext` con buffers locales:

```
struct WorkerContext {
    std::size_t thread_id{0};
    Simulator *sim{nullptr};
    std::uint64_t dead_rabbits{0}, eaten_rabbits{0}, dead_foxes{0};
    std::vector<Entity*> planned_rabbits, planned_foxes;
    std::vector<std::pair<std::size_t, std::size_t>> rabbit_new_pos, fox_new_pos;
    std::vector<std::unique_ptr<Entity>> local_new_entities; // Nacimientos locales
    std::vector<Entity*> local_new_rabbits, local_new_foxes;
};
```

4.3. Determinismo Absoluto en el Arbitraje

En sistemas paralelos, el orden en que los hilos obtienen un cerrojo (lock) es no determinista. Si dos animales con igual `proc_age` compitieran por la misma celda, el resultado dependería del planificador del sistema operativo. Para resolver esto de raíz y garantizar **exacta reproducibilidad frente al código secuencial**, se incorporó un desempate total por el identificador único (`id`):

```
// Arbitraje determinista de conejos:
bool rabbit_wins = (rabbit->proc_age > occupant->proc_age) ||
                   (rabbit->proc_age == occupant->proc_age && rabbit->id < occupant->id);

// Arbitraje determinista de zorros:
bool fox_wins = (fox->proc_age > occupant->proc_age) ||
                (fox->proc_age == occupant->proc_age && fox->starve_age < occupant->starve_age) ||
                (fox->proc_age == occupant->proc_age && fox->starve_age == occupant->starve_age &&
                 fox->id < occupant->id);
```

5. Comparativa Técnica: Secuencial vs. Paralelo

Criterio	Versión Secuencial	Versión Paralela (Pthreads)
Concurrencia	1 hilo de ejecución (monohilo).	4 hilos fijos en Thread Pool persistente.
Cálculo de Desplazamiento	Bucle `for` lineal sobre todo el vector de entidades.	Partición balanceada en 4 rangos disjuntos calculados en paralelo.
Escritura en `next_grid`	Acceso directo sin protección.	Exclusión mutua granular por filas (`row_mutexes[y]`).
Gestión de Nacimientos	`entities.push_back()` directo al vector principal.	Buffers locales en `WorkerContext` consolidados al final del turno.
Contadores Poblacionales	Incrementos directos en variables miembro.	Acumuladores locales por hilo consolidados sin contención de caché.
Complejidad Espacial Extra	$O(1)$ adicional.	$O(R)$ mutexes de fila + buffers locales de 4 hilos.
Determinismo de Salida	100% determinista.	100% determinista e idéntico a la versión secuencial.

6. Validación y Resultados de Ejecución

6.1. Validación con el Caso Oficial de Prueba (5x5 UVA)

Se verificó rigurosamente la precisión numérica y la lógica de arbitraje ejecutando el caso oficial provisto en el enunciado con parámetros: `GEN_PROC_RABBITS = 2` , `GEN_PROC_FOXES = 4` , `GEN_FOOD_FOXES = 3` , `N_GEN = 6` , dimensiones 5×5 y 9 entidades iniciales.

Entrada Oficial:

```
2 4 3 6 5 5 9
ROCK 0 0
RABBIT 0 2
FOX 0 4
FOX 1 0
FOX 1 4
ROCK 2 4
RABBIT 3 0
RABBIT 4 0
FOX 4 4
```

Salida Esperada y Obtenida:

```
2 4 3 0 5 5 5
ROCK 0 0
ROCK 2 4
RABBIT 3 0
FOX 4 0
FOX 4 3
```

La salida producida por la versión paralela con Pthreads coincide **byte a byte** con la salida de referencia, validando la exactitud del movimiento simultáneo y las reglas de depredación.

6.2. Rendimiento en la Aplicación Interactiva (Raylib)

Al ejecutar el entorno gráfico interactivo (`interactiva.exe`), el hilo de la interfaz de usuario delega de forma asíncrona y continua el avance de cada fotograma al Thread Pool mediante `simulator->update()` . Se mantiene una tasa estable de **60 cuadros por segundo** incluso con densidades elevadas de población y grillas de alta resolución, demostrando la ausencia de latencias por creación de hilos y la eficacia del bloqueo granular.

7. Conclusiones

1. **Aislamiento Arquitectónico Exitoso:** Se logró paralelizar íntegramente la lógica del simulador dentro de `Simulator::update()` sin modificar una sola línea de código en los módulos `Grid` , `Entity` , `main.cpp` o `interactiva.cpp` .
2. **Eficiencia del Thread Pool:** Mantener los 4 hilos en estado persistente con barreras de sincronización elimina el costo computacional de creación/destrucción recurrente de hilos en cada generación.
3. **Escalabilidad del Bloqueo Granular:** El uso de un mutex por fila en lugar de un mutex global permite que el arbitraje espacial escale proporcionalmente con el número de núcleos sin generar serialización.
4. **Determinismo Concurrente:** La regla de arbitraje extendida con desempate por identificador asegura que la simulación paralela produzca siempre resultados 100% reproducibles y equivalentes al modelo secuencial formal.